

The Vanka Smoother

Theory, and a high-performance implementation for the SFEM
semi-structured matrix-free CVFEM Navier–Stokes solver

September 15, 2026

Why this document exists

Measured on our solver, the block-Jacobi smoother has an asymptotic convergence factor of $\rho \approx 0.981$ at $\text{Re} = 1$ — 1.9% error removal per sweep, 120 sweeps per decade — and *diverges* for $\omega \geq 0.5$. The SIMPLE variant measures identically. Local Fourier analysis for Stokes puts a Vanka smoother at $\rho = 0.08$.

That difference, roughly a factor of 50 in iteration count, is the single largest lever available to this solver.

1 What Vanka is, and why block-Jacobi fails here

1.1 The saddle-point structure

Our linearised system has the block form

$$\mathcal{A} = \begin{pmatrix} F & B^\top \\ B & -C \end{pmatrix}, \quad \begin{pmatrix} \delta u \\ \delta p \end{pmatrix}, \quad (1)$$

where F is the momentum operator (viscous plus convective), B the discrete divergence, $B^\top$ the pressure gradient, and C the Rhie–Chow stabilisation. Without stabilisation $C = 0$ and the system is a genuine saddle point.

A *point-block* smoother inverts, at each node i , the 4×4 block

$$D_i = \begin{pmatrix} F_{ii} & B_{ii}^\top \\ B_{ii} & -C_{ii} \end{pmatrix}, \quad (2)$$

and this is where it fails. The pressure row of D_i contains only the *local* part of the divergence constraint. Continuity is a statement about fluxes across the faces *surrounding* a node, so B_{ii} carries almost none of it: the constraint that actually determines the pressure lives in the off-diagonal couplings B_{ij} , $j \neq i$, which the point-block smoother discards entirely. Damping the momentum equation while ignoring the constraint is why $\rho \rightarrow 1$: the velocity error is smoothed, and the pressure error is barely touched.

1.2 The Vanka idea

Vanka’s construction (S. P. Vanka, *JCP* **65** (1986) 138–158) replaces the per-node block by a small *coupled local system* over a patch Ω_k of degrees of freedom chosen so that at least one pressure unknown appears *together with all the velocity unknowns that its continuity equation references*. Writing R_k for the restriction onto patch k , one sweep of additive Vanka is

$$x \leftarrow x + \omega \sum_k R_k^\top (R_k \mathcal{A} R_k^\top)^{-1} R_k (b - \mathcal{A}x). \quad (3)$$

The inverted object $\mathcal{A}_k = R_k \mathcal{A} R_k^\top$ is the true operator restricted to the patch, *including* the velocity–pressure coupling. That is the whole difference from block-Jacobi, and it is why the local solve can enforce the constraint.

Published rates (Stokes, LFA)

Smoother	ρ (periodic)	ρ (Dirichlet)
Vanka	0.08	0.10
Triad Gauss–Seidel	0.26	0.36
Triad Jacobi	0.49	divergent
<i>our point-block Jacobi (measured, Re=1)</i>	—	<i>0.981</i>

Claus & Bolten, [arXiv:2008.08719](https://arxiv.org/abs/2008.08719). Our smoother is weaker than the entry that study calls divergent, under the boundary conditions we run.

2 Choosing the patch for a colocated CVFEM discretisation

Vanka was devised for staggered (MAC) grids, where the patch is obvious: one cell-centred pressure plus the six face velocities that its continuity equation uses. Our discretisation is *colocated* — all four fields live at every node — so the patch must be chosen deliberately.

Patch	Size	Comment
Node star	$1 + 27 \times 3 = 82$	one pressure, velocities of the 3^3 neighbourhood
Micro-element	$8 \times 4 = \mathbf{32}$	the 8 corners of one micro-cell, all fields
Pressure Schur only	8	micro-element patch with velocities eliminated

Recommendation: the micro-element patch

The micro-element patch is the right choice here for a reason specific to our code: the 32×32 patch operator $\mathcal{A}_k$ is *exactly the dense micro-cell matrix we already know how to build*. Passing identity slots to `cvfem_hex8_ns_upwind_jacobian_add_slots` writes it into a local buffer — the same trick `assemble_block_diag` uses and the same one the element-wise Galerkin assembly relies on. No new kernel is needed to form the patch operator.

It also aligns with every structure the solver already has: the micro-cell loop, the constant corner offsets on the lattice, and the deterministic two-pass scatter.

A subtlety in our favour: because Rhie–Chow supplies $C \neq 0$, the patch matrix has a non-zero pressure block and is nonsingular without any extra pinning. On an unstabilised saddle point the local systems are singular up to a constant and need care; we avoid that.

3 Variants, and what each costs

3.1 Full versus diagonal Vanka

Full Vanka inverts $\mathcal{A}_k$ exactly. *Diagonal* Vanka approximates the velocity block F_k by its diagonal, which lets the velocities be eliminated analytically and leaves a small pressure system — the local Schur complement

$$S_k = -C_k - B_k \operatorname{diag}(F_k)^{-1} B_k^\top, \quad (4)$$

of size 8×8 for the micro-element patch. This is the variant most production codes use; it retains the essential velocity–pressure coupling while removing most of the cost.

3.2 Cost model

At $L = 8$ with 108 macro-elements: 55 296 micro-cells, 60 625 nodes, 242 500 dof.

Variant	patch n	factor (flop)	apply (flop)	factors, all cells
Full Vanka	32	21 845	2048	432 MiB
Diagonal Vanka	8	341	128	27 MiB

Per-sweep cost of the local solves alone, against the current smoother:

nodal 4×4 block-Jacobi (current)	1.94 MFLOP
diagonal Vanka, 8×8 Schur	7.08 MFLOP
full Vanka, 32×32	113.25 MFLOP

The trade, stated plainly

The local solve is a small part of a sweep — the sweep is dominated by the residual evaluation, which is unchanged. Diagonal Vanka should raise sweep cost by tens of percent, not multiples. Against that, moving ρ from 0.98 to even 0.3 is a factor of ~ 50 in iteration count. Full Vanka’s 432 MiB of cached factors is the reason to start with the diagonal variant, not its flop count.

4 High-performance implementation on the semi-structured mesh

4.1 Additive versus multiplicative, and colouring

Equation (3) is *additive*: every patch reads the same residual, so all patches are independent and the sweep is embarrassingly parallel. It converges more slowly than the *multiplicative* form, in which each patch sees the updates of those before it (Gauss–Seidel in patches).

The multiplicative form parallelises by colouring patches so that same-coloured patches share no degrees of freedom. On our lattice this is exact and free: two micro-cells share a node iff their indices differ by at most one in every axis, so the parity classes

$$c(i, j, k) = (i \bmod 2) + 2(j \bmod 2) + 4(k \bmod 2) \in \{0, \dots, 7\} \quad (5)$$

give **eight colours**, and within a colour every pair of cells differs by at least two in some axis and is therefore independent. Eight sequential passes, each fully parallel over $L^3/8$ cells per macro-element.

4.2 Accumulation and determinism

Additive Vanka writes overlapping contributions to shared nodes — exactly the pattern the solver already handles. Reuse **SSScatter**: each macro-element writes its exclusively owned nodes directly and stages the shared ones, then a second pass sums each shared node’s contributions in a fixed order. No atomics, and the sweep is bitwise reproducible on any thread count, which the existing determinism gates will check for free.

4.3 Where the factorisations live

The patch matrices depend on the linearisation state, so they are valid for one Newton step and must be rebuilt when it changes — the same lifetime as the element-wise Galerkin coarse operators, and they can be built in the same pass.

- **Cache the 8×8 Schur factors** (27 MiB at $L = 8$): build once per Newton step, apply on every sweep. Six sweeps per V-cycle and hundreds of cycles per Newton step make this overwhelmingly the right trade.
- Store them in the *lattice* layout already used elsewhere — indexed by `(macro element, micro-cell)` — so the sweep streams them contiguously with no indirection, exactly as the 27-point stencil coarse operators do.
- Do *not* cache the full 32×32 factors; 432 MiB is not worth it before the diagonal variant has been measured.

4.4 Matrix-free, precisely

“Matrix-free” here means no *global* sparse matrix, not no local matrices. Vanka needs the patch entries, and we form them transiently per micro-cell from the existing kernel, use them, and discard them — which is what the element-wise Galerkin assembly already does. The fine-level residual evaluation, which dominates the sweep, stays exactly as it is.

5 Implementation plan

1. **Form the patch operator.** Per micro-cell, identity slots into `cvfem_hex8_ns_upwind_jacobian_add_slots` give the dense 32×32 $\mathcal{A}_k$. Already available; no new kernel.
2. **Build the local Schur complement.** Extract $\text{diag}(F_k)$, form $S_k = -C_k - B_k \text{diag}(F_k)^{-1} B_k^\top$ (8×8), factor it, and store by `(element, cell)`. One pass per Newton step, fusable with the Galerkin assembly pass.
3. **Additive sweep first.** Residual, then per cell: gather 32 local values, solve S_k for the pressure correction, back-substitute the velocities, scatter through `SSScatter`. Additive is simpler and its correctness is easier to establish.
4. **Measure with SFEM.GMG_CHECK=3**, 150 sweeps, to the asymptote — the same gate that produced the 0.981 figure, so the comparison is like for like. Sweep ω .
5. **Then multiplicative with 8-colour parity** if the additive rate is insufficient. Expect a substantially better factor at the cost of eight sequential passes.
6. **Only then reconsider full Vanka**, and only if the diagonal variant’s rate is close enough to make the memory worth it.

Acceptance criterion

The measurement that matters is the standalone smoother rate at the *physical* Reynolds number, run to its asymptote with no coarse space. Block-Jacobi gives 0.981 at $\text{Re} = 1$. Any Vanka variant that does not comfortably beat that, at $\text{Re} \geq 100$, is not worth carrying — and a rate near or above 1 must be treated as disqualifying however good it looks over the first few sweeps, because the rate can fall for twenty sweeps and then climb through unity.

6 The smoother as implemented: set-up and apply

The plan above is implemented in `src/ss/cvfem_ss_vanka.hpp`: diagonal Vanka on the micro-element patch, multiplicative inside a macro-element and additive between macro-elements, used on every smoothed multigrid level. It differs from the plan in two ways. First, the patch operator

is read from the *assembled* operator, not from one micro-cell's contribution. Second, the coarse levels run the same code on the Galerkin operators the hierarchy already builds. This section gives the set-up and the apply at the level of detail needed to reason about their cost or to reimplement them.

6.1 Notation and data

Lattice. There are n_M macro-elements at level L . Each has $(L+1)^3$ lattice nodes and L^3 micro-cells. Lattice node (x, y, z) has local index $\ell(x, y, z) = x + (L+1)(y + (L+1)z)$, and $g_e(\ell)$ is its global node in macro-element e . The eight corners of micro-cell (x, y, z) are $\ell(x, y, z) + o_a$ with

$$o = (0, 1, L+2, L+1, (L+1)^2, (L+1)^2+1, (L+1)^2+L+2, (L+1)^2+L+1). \quad (6)$$

Operator. A is stored in BSR with 4×4 blocks, sorted column indices, and at most 27 blocks per row. Fields are ordered (u_x, u_y, u_z, p) and each block is row-major. For nodes i, j the block splits as

$$A_{ij} = \begin{pmatrix} F_{ij} & G_{ij} \\ D_{ij} & A_{ij}^{pp} \end{pmatrix}, \quad F_{ij} \in \mathbb{R}^{3 \times 3}, G_{ij} \in \mathbb{R}^3, D_{ij} \in \mathbb{R}^{1 \times 3}, A_{ij}^{pp} = -C_{ij}. \quad (7)$$

G is the pressure gradient ($B^\top$ above) and D the divergence (B). The patch operator of cell k , with corner nodes $n_1, \dots, n_8$, is the 32×32 matrix of blocks $A_{n_a n_b}$. On the fine level, A comes from the element-wise Galerkin assembly at ratio $q = 1$. There the prolongation is the identity and $P^\top A P = A$, and that patch matches the matrix-free apply to 1.54×10^{-16} . On a coarse level, A is that level's Galerkin coarse operator. Each level passes its own mask of constrained degrees of freedom.

Why the assembled operator. A node inside a macro-element lies in eight micro-cells, so a single cell's element matrix carries about an eighth of the true patch entries. A smoother built on it diverged at a rate of 18. Taking only the diagonal from the assembled matrix and the off-diagonal entries from the cell made the patch system internally inconsistent, and that version diverged at 7.8.

Per-cell record. Set-up stores the following per micro-cell, in lattice order (e, z, y, x) , so the sweep streams the records contiguously. T is the storage type (§6.7).

Field	Shape	Contents	Used for
$\hat{D}$	8×24	$D_{n_a n_i}[t]/F_{n_i n_i}[t, t]$	pressure right-hand side
$\hat{G}$	24×8	$G_{n_i n_b}[t]/F_{n_i n_i}[t, t]$	velocity back-substitution
d^{-1}	24	$1/F_{n_i n_i}[t, t]$ (floored)	velocity back-substitution
S, π	$8 \times 8, 8$	LU factors and pivots of the Schur complement	pressure solve

That is 472 values of type T plus 8 integers: 1 920 B in single precision and 3 808 B in double. At $L = 8$ with 420 macro-elements there are 215 040 cells, which is 394 MiB in single precision and 781 MiB in double.

6.2 Set-up

Set-up runs once per Newton step, because the patch operators depend on the linearisation. This is the same lifetime as the Galerkin coarse operators.

Algorithm 1 Vanka set-up, `vanka_setup`

Require: assembled BSR operator A ; constraint mask; tolerances $\tau_d = \tau_S = 10^{-14}$

```
1:  $w_i \leftarrow 1/\#\{\text{micro-cells containing node } i\}$  ▷ additive form
2:  $w_i^E \leftarrow 1/\#\{\text{macro-elements containing node } i\}$  ▷ multiplicative form
3: for all macro-elements  $e$  in parallel do
4:   for all micro-cells  $k = (x, y, z)$  of  $e$  do
5:      $n_a \leftarrow g_e(\ell(x, y, z) + o_a), \quad a = 1, \dots, 8$ 
6:     for  $a, b = 1, \dots, 8$  do ▷ 64 blocks, double precision
7:        $A[a][b] \leftarrow A_{n_a n_b}$ , found by binary search in row  $n_a$  (zero if absent)
8:     end for
9:      $s \leftarrow \max_{a,t} |F_{n_a n_a}[t, t]|$  ▷ the patch's own velocity scale
10:    for  $a = 1, \dots, 8, t = 1, 2, 3$  do
11:       $\delta \leftarrow F_{n_a n_a}[t, t]$ 
12:       $d_{a,t}^{-1} \leftarrow 1/\delta$  if  $|\delta| > \tau_d s$  else 0
13:    end for
14:     $\hat{D}_{a,(i,t)} \leftarrow D_{n_a n_i}[t] d_{i,t}^{-1}, \quad \hat{G}_{(i,t),b} \leftarrow G_{n_i n_b}[t] d_{i,t}^{-1}$ 
15:     $S_{ab} \leftarrow A_{n_a n_b}^{pp} - \sum_{i=1}^8 \sum_{t=1}^3 D_{n_a n_i}[t] d_{i,t}^{-1} G_{n_i n_b}[t]$ 
16:    factor  $S = P^T LU$  with partial pivoting; reject pivot  $\leq \tau_S \max_{ab} |S_{ab}|$ 
17:    if factorisation rejected then
18:       $S \leftarrow I, \quad \pi \leftarrow \text{id}$  ▷ withhold this patch's pressure update
19:    end if
20:    narrow  $\hat{D}, \hat{G}, d^{-1}, S$  to  $T$  and store in the record of  $k$ 
21:  end for
22: end for
23: if  $T$  is narrower than the assembled values then
24:   allocate  $\tilde{A}$  uninitialised and fill  $\tilde{A} \leftarrow (T) A$  in parallel ▷ first touch by the sweeping threads
25: else
26:    $\tilde{A} \leftarrow A$  ▷ alias, no copy
27: end if
```

The safeguards. Each one records a failure that was observed:

- *The velocity-diagonal floor is relative.* An absolute floor of 10^{-300} let a diagonal of 3.6×10^{-17} through at a do-nothing outflow node, where the mass flux across the face is near zero. The resulting $d^{-1} = 2.8 \times 10^{16}$ was exactly the per-sweep amplification of the unstable smoother, and it was independent of ω . The floor is far below any legitimate value: at 10^{-11} it zeroed healthy diagonals and took Poiseuille flow from 178 to 1 119 linear iterations. The optional `SFEM.VANKA_ROW_FRAC` bounds $|\delta|$ below by a fraction of the row's ℓ^1 norm over the patch velocities. It is off by default and bit-identical when off.
- *The pivot tolerance is relative.* It is measured against the Schur complement's own largest entry, for the same reason. A pivot of 10^{-20} in a complement of scale 10^{-3} once returned values of about 10^{17} from the preconditioner.
- *A rejected patch falls back to the identity.* This makes the smoother weaker for that patch rather than explosive.
- *Arithmetic is in double.* Every product and the factorisation are computed in double precision, and results are narrowed to T only when stored.

Cost. Each cell does 64 binary searches over at most 27 columns, forms the Schur complement with $8 \cdot 8 \cdot 24 = 1\,536$ triple products, and factors an 8×8 (≈ 170 multiply-adds). The loop has

no shared writes and is parallel over macro-elements. On the FDA nozzle at 893 924 dof ($L = 8$, 215 040 cells, 72 Grace cores), the fine-level set-up takes 5.1 s over 28 Newton steps, which is 0.18 s per step and 8 % of the run.

6.3 The patch solve

The additive and multiplicative applies share one local solve. Given the patch residual $r_u \in \mathbb{R}^{24}$ and $r_p \in \mathbb{R}^8$, eliminating the velocities through $\text{diag}(F_k)$ gives

$$S \delta p = r_p - \hat{D} r_u, \quad \delta u = d^{-1} \odot r_u - \hat{G} \delta p. \quad (8)$$

Algorithm 2 Patch solve

Require: cell record $(\hat{D}, \hat{G}, d^{-1}, S, \pi)$; r_u, r_p in double

- | | |
|---|---------------------|
| 1: $\delta p_a \leftarrow r_{p,a} - \sum_{j=1}^{24} \hat{D}_{aj} r_{u,j}$ | ▷ 192 multiply-adds |
| 2: apply π to δp ; forward-substitute with L ; back-substitute with U | ▷ 64 |
| 3: $\delta u_j \leftarrow d_j^{-1} r_{u,j} - \sum_{b=1}^8 \hat{G}_{jb} \delta p_b$ | ▷ 216 |
-

The stored values are promoted to double as they are read, so the local arithmetic is double whatever T is.

6.4 Additive apply

Algorithm 3 Additive Vanka, vanka_apply: $y \leftarrow y + \omega W \sum_k R_k^\top \hat{A}_k^{-1} R_k r$

- 1: $z \leftarrow 0$
 - 2: **for all** macro-elements e **in parallel do**
 - 3: gather $r_\ell \leftarrow r_{g_e(\ell)}$ for all $(L+1)^3$ lattice nodes; $z^e \leftarrow 0$
 - 4: **for all** micro-cells k of e **do**
 - 5: restrict r^e to the 8 corners $\rightarrow (r_u, r_p)$
 - 6: $(\delta u, \delta p) \leftarrow \text{Algorithm 2}$
 - 7: $z_{\ell+o_a}^e += (\delta u_a, \delta p_a)$ for $a = 1, \dots, 8$
 - 8: **end for**
 - 9: two-pass scatter, pass 1: write z^e to nodes owned by e alone; stage shared nodes
 - 10: **end for**
 - 11: two-pass scatter, pass 2: sum staged contributions per shared node in a fixed order
 - 12: **for all** nodes i , fields t , not constrained **do**
 - 13: $y_{i,t} += \omega w_i z_{i,t}$
 - 14: **end for**
-

Every patch reads the same residual, so the cells are independent. The weight w_i averages over the patches containing a node. Without it, a node inside a macro-element would receive eight times the correction of a lattice corner.

6.5 Multiplicative apply

This is the default (SFEM_VANKA_MULT=1). Standalone at $\text{Re} = 1$ it measures $\rho = 0.63$ at $\omega = 1$, against 0.883 for the additive form.

Algorithm 4 Multiplicative Vanka, `vanka_apply_mult`

```
1:  $z \leftarrow 0$ 
2: for all macro-elements  $e$  in parallel do ▷ thread-private  $\lambda$ , initially  $-1$ 
3:   for  $\ell = 1, \dots, (L+1)^3$  do
4:      $\lambda(g_e(\ell)) \leftarrow \ell$ ;  $r_\ell^e \leftarrow r_{g_e(\ell)}$ 
5:   end for
6:    $z^e \leftarrow 0$ 
7:   for colour  $c = 0, \dots, 7$  do ▷  $c = c_x + 2c_y + 4c_z$ 
8:     for all micro-cells  $k = (x, y, z)$  with  $(x, y, z) \equiv (c_x, c_y, c_z) \pmod{2}$  do
9:       for  $a = 1, \dots, 8$  do ▷ local residual from the current local iterate
10:         $\ell_a \leftarrow \ell(x, y, z) + o_a$ ,  $\rho \leftarrow r_{\ell_a}^e$ 
11:        for all blocks  $(j, \tilde{A}_{g_e(\ell_a)j})$  in BSR row  $g_e(\ell_a)$  do
12:          if  $\lambda(j) \geq 0$  then
13:             $\rho \leftarrow \rho - \tilde{A}_{g_e(\ell_a)j} z_{\lambda(j)}^e$ 
14:          end if ▷ columns outside  $e$  are skipped
15:        end for
16:         $(r_{u,a}, r_{p,a}) \leftarrow \rho$ 
17:      end for
18:       $(\delta u, \delta p) \leftarrow \text{Algorithm 2}$ 
19:       $z_{\ell_a}^e += \omega(\delta u_a, \delta p_a)$  for  $a = 1, \dots, 8$ 
20:    end for
21:  end for
22:  two-pass scatter, pass 1 of  $z^e$ 
23:  for  $\ell = 1, \dots, (L+1)^3$  do
24:     $\lambda(g_e(\ell)) \leftarrow -1$  ▷ reset costs  $(L+1)^3$ , not  $n_{\text{nodes}}$ 
25:  end for
26: end for
27: two-pass scatter, pass 2
28: for all nodes  $i$ , fields  $t$ , not constrained do
29:    $y_{i,t} += w_i^E z_{i,t}$ 
30: end for
```

Why multiplicative only inside a macro-element. The eight-colour argument holds inside one macro-element: two micro-cells there share a node exactly when their indices differ by at most one on every axis. It fails across macro-elements, where same-coloured cells share face nodes. A sweep that ran colours globally in parallel over macro-elements raced on those nodes, and the determinism gate caught it: 36 linear iterations on one thread against 156 on eight. Each macro-element therefore sweeps its own cells in Gauss–Seidel order on a private vector. Its update reads only its own nodes, because the map λ drops every column outside e . The corrections are then combined additively through the two-pass scatter and averaged with w^E . There are no atomics, and the result is bit-identical on any thread count.

Cost, and why it is memory bound. The patch solve is about 470 multiply-adds per cell. The local residual walks 8 BSR rows of up to 27 blocks, which is up to $8 \cdot 27 \cdot 16 = 3456$ multiply-adds and 216 indirect lookups per cell, reading 64 bytes per block in single precision. The row walk is therefore the larger cost, and it reads memory in a pattern the prefetcher does not follow. On the nozzle at 893 924 dof the fine-level sweep was 72 % of the socket’s cycles, and 72 % of those were stalled in the back end. The bytes streamed, not the flop count, set the speed of the sweep.

6.6 How the multigrid cycle calls it

Each smoothed level wraps the smoother P (Algorithm 4, or 3) in a stationary iteration with $\nu = 2$ sweeps (`SFEM_GMG_SMOOTH`), before and after the coarse correction. The coarsest level is solved by dense LU. Because P accumulates into its output,

$$x^{(m+1)} = x^{(m)} + P(b - Ax^{(m)}), \quad m = 0, \dots, \nu - 1. \quad (9)$$

The first pre-smoothing sweep of a cycle always starts from $x^{(0)} = 0$: FGMRES passes a zero vector to the preconditioner, and coarse levels start from zero by construction. In that case the iteration is told so (`set_initial_guess_zero`) and takes $x^{(1)} = Pb$ directly. This saves one operator application per level per cycle and gives the same bits.

The defaults are $\omega = 1$ (`SFEM_VANKA_OMEGA`) with the multiplicative form. An open do-nothing outlet changes this: inside the cycle it sets $\omega = 0.5$, and without multigrid it selects the additive form. An explicitly set variable always takes precedence. On the fine level, the set-up assembles A through the Galerkin path at $q = 1$ (`make_diagonal_vanka`). On a coarse level it reads the Galerkin BSR that level already holds (`make_diagonal_vanka_from_bsr`).

6.7 Storage precision

The storage type T of the cell records and of the row walk’s values $\tilde{A}$ is set by the environment variable `SFEM_VANKA_PRECISION`. It is either `single` (the default, $T = \text{float}$) or `double`. Both compute in double, and both are the same templated code. Single is the default because it measures faster. Measured on the warped FDA nozzle at throat $\text{Re} = 500$, FGMRES with the geometric multigrid, 72 Grace cores. Every run had the same Newton and linear iterations, flux and station velocities:

dof	storage	fine-level smoothing	run
116 212	single	1.98 s	7.23 s
116 212	double	2.40 s	7.82 s
893 924	single	40.2 s	65.8 s
893 924	double	70.1 s	96.6 s

At 893 924 dof one fine-level smoothing step costs 17.4 ms in single precision and 30.4 ms in double. A step is two sweeps plus the operator application between them, and that operator application does not change with T . The factor of 1.75 over the whole step is therefore a lower bound for the sweeps alone. It is close to the factor of two in bytes streamed, which is further evidence that the sweep is memory bound. A smoother only has to reduce the error, and FGMRES absorbs the difference.

7 Is Vanka a Navier–Stokes smoother, or only a Stokes one?

It is a Navier–Stokes smoother by origin: Vanka’s 1986 papers solve the primitive-variable Navier–Stokes equations on recirculating cavity flows, not Stokes, and Vanka-type smoothers are standard in finite-element Navier–Stokes multigrid. The “Stokes smoother” impression comes from the *analysis*, not the method — local Fourier analysis needs a linear, constant-coefficient operator, so published LFA rates are almost always Stokes rates.

What the 0.08 does and does not mean

The LFA rates quoted in this document measure how well the smoother handles the **saddle-point coupling**. They say nothing about **convection dominance**, which is a second and independent difficulty:

- At low Re the momentum block F is essentially elliptic, and a local patch solve is a good local approximation — the regime the LFA describes.
- As Re rises, F becomes convection-dominated and nearly hyperbolic. Information travels along streamlines, which is exactly what a *local* smoother cannot follow. The classical remedies are flow-aligned (downstream) ordering of the multiplicative sweep, or line and plane smoothers, or semi-coarsening — not a larger patch.

7.1 Why it is still the right first step here

Our measured failure is at $\text{Re} = 1$: $\rho = 0.981$ in what is essentially the Stokes limit. That is a pure saddle-point-coupling failure, precisely the thing Vanka fixes, and it has to be fixed before Reynolds robustness is a meaningful question at all. Two separate problems are being conflated if we skip it:

Symptom	Regime	Remedy
$\rho = 0.981$ at $\text{Re} = 1$	Stokes limit	Vanka (this document)
divergence at $\text{Re} \geq 400$	convection-dominated	AL preconditioner, or ordering/semi-coarsening

7.2 What Vanka is unlikely to do

It should not be assumed that Vanka alone reaches $\text{Re} \geq 400$. For Reynolds robustness the reference result is the augmented-Lagrangian preconditioner of Farrell, Mitchell & Wechsung (arXiv:1810.03315), which holds 4–8 Krylov iterations per Newton step to $\text{Re} = 10\,000$. That is not a drop-in replacement for a smoother: the augmented term $\gamma B^\top W^{-1} B$ makes the momentum block nearly singular, with a large kernel, and their multigrid uses a *kernel-capturing subspace-correction* smoother built for that structure rather than Vanka.

The realistic plan is therefore two-stage: Vanka to recover a working smoother in the Stokes-limit sense, then reassess Reynolds robustness on the evidence, with augmented Lagrangian as the expected second step if the Re sweep still degrades.

8 Risks and open questions

- **Colocated patches are not the classical construction.** Vanka’s rates are published for staggered discretisations. The micro-element patch is the natural colocated analogue, but the LFA numbers do not transfer automatically and our own measurement is the only evidence that will count.
- **Rhie–Chow coupling.** $C \neq 0$ makes the patches nonsingular, which helps, but the stabilisation scale $D_f = \text{rc } h^2 / (2\mu)$ grows as Re rises and as the mesh coarsens; the patch conditioning should be checked across the hierarchy, not only on the fine level.
- **Convection dominance** — see the section above. No Stokes rate speaks to it, and measuring the Reynolds dependence of whatever smoother we adopt is not optional.
- **Coarse levels.** The coarse levels are assembled BSR, not matrix-free, so their patch operators must be extracted from the assembled matrix instead of from the cell kernel — a second, simpler code path.